

Step 1 – Download all the files from [Kaggle Link](#)

Step 2 – Upload to GitHub account

Adventure-Works (Public) Pin Watch 0

main 1 Branch 0 Tags Go to file T Add file <> Code

sreemala85 Add files via upload 2283f38 · 3 days ago 2 Commits

File	Action	Time
AdventureWorks_Calendar.csv	Add files via upload	3 days ago
AdventureWorks_Customers.csv	Add files via upload	3 days ago
AdventureWorks_Product_Categories.csv	Add files via upload	3 days ago
AdventureWorks_Product_Subcategories.csv	Add files via upload	3 days ago
AdventureWorks_Products.csv	Add files via upload	3 days ago
AdventureWorks>Returns.csv	Add files via upload	3 days ago
AdventureWorks_Sales_2015.csv	Add files via upload	3 days ago
AdventureWorks_Sales_2016.csv	Add files via upload	3 days ago
AdventureWorks_Sales_2017.csv	Add files via upload	3 days ago

Step 3 – Creating Azure resources in MS Azure (BRONZE LAYER)

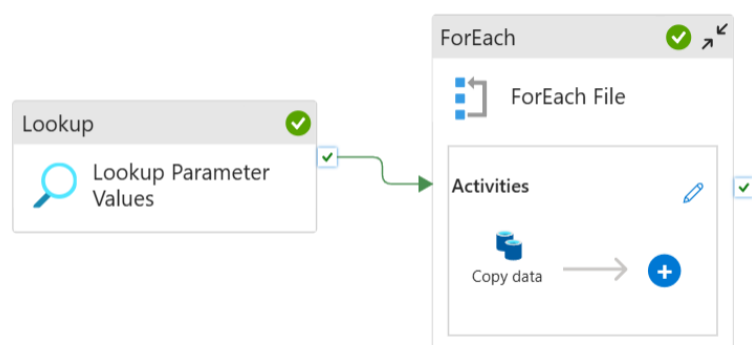
- Create a Resource Group
- Create Data Lake/ Storage Account
- Create containers inside the storage account (*Bronze, Silver*)
- Create ADF resource
- Create Databricks Resource

Step 4 – Launch ADF workspace to ingest raw data from Github to ADLS

Source – HTTP

Destination – ADLS

a. Pipeline Structure



b. Pipeline Overview –

- > **Lookup activity** will retrieve the parameter values written in JSON format.
- > Based on each parameter value, it will iterate each folder (**ForEach iterator**) and file in ADLS.
- > **Copy Data activity** copies the file from Github to ADLS in bronze layer.

c. Detailed steps for pipeline creation

i. Create a **JSON file** – *paramvalues* which contains all the parameter values and upload the file to bronze container in ADLS.

```
[
  {
    "p_rel_url" : "Adventure-Works/refs/heads/main/AdventureWorks_Calendar.csv",
    "p_sink_folder" : "Calendar",
    "p_sink_file_name" : "Calendar.csv"
  },
  {
    "p_rel_url" : "Adventure-Works/refs/heads/main/AdventureWorks_Customers.csv",
    "p_sink_folder" : "Customers",
    "p_sink_file_name" : "Customers.csv"
  },
  {
    "p_rel_url" : "Adventure-Works/refs/heads/main/AdventureWorks_Product_Categories.csv",
    "p_sink_folder" : "Product_Categories",
    "p_sink_file_name" : "Product_Categories.csv"
  },
  {
    "p_rel_url" : "Adventure-Works/refs/heads/main/AdventureWorks_Product_Subcategories.csv",
    "p_sink_folder" : "Product_Subcategories",
    "p_sink_file_name" : "Product_Subcategories.csv"
  }
],
```

ii. Configure Lookup activity

General **Settings** User properties

Source dataset * JSON_Parquet Open New Preview data

First row only ☐

File path type ☒ File path in dataset ☐ Wildcard file path ☐ List of files ⓘ

Filter by last modified ⓘ Start time (UTC) End time (UTC)

Recursively ⓘ ☒

Enable partitions discovery ⓘ ☐

Max concurrent connections ⓘ

iii. Configure ForEach iterator

General **Settings** Activities (1) User properties

Sequential ☒

Items @activity('Lookup Parameter Values')...

iv. Configure Copy Data activity

General **Source** Sink Mapping Settings User properties

Source dataset * ds_http Open New Preview data

Dataset properties ⓘ

Name	Value
param_rel_url	@item().p_rel_url

Request method * ⓘ GET

GeneralSourceSinkMappingSettingsUser properties

Sink dataset *

ds_adls

OpenNewLearn more

Dataset properties

Name	Value
param_sinkfolder	@item().p_sink_folder
param_sinkfilename	@item().p_sink_file_name

v. Run the pipeline. Publish

ParametersVariablesSettingsOutput

Pipeline run ID110813af-f6bb-4d40-93fd-2551337453e

Pipeline status

Succeeded

View debug run c

All statusList

Monitor in Azure MetricsExport t

Showing 1 - 12 of 12 items

Activity name	Activity st...	Activit...	Run start	Duration	Integration runtime
Copy data	Succeeded	Copy data	6/14/2026, 6:12:24 AM	18s	AutoResolveIntegrationRuntime (UAE North)
Copy data	Succeeded	Copy data	6/14/2026, 6:12:06 AM	16s	AutoResolveIntegrationRuntime (UAE North)
Copy data	Succeeded	Copy data	6/14/2026, 6:11:49 AM	17s	AutoResolveIntegrationRuntime (UAE North)
Copy data	Succeeded	Copy data	6/14/2026, 6:11:33 AM	14s	AutoResolveIntegrationRuntime (UAE North)
Copy data	Succeeded	Copy data	6/14/2026, 6:11:17 AM	15s	AutoResolveIntegrationRuntime (UAE North)
Copy data	Succeeded	Copy data	6/14/2026, 6:11:01 AM	16s	AutoResolveIntegrationRuntime (UAE North)
Copy data	Succeeded	Copy data	6/14/2026, 6:10:45 AM	15s	AutoResolveIntegrationRuntime (UAE North)
Copy data	Succeeded	Copy data	6/14/2026, 6:10:27 AM	17s	AutoResolveIntegrationRuntime (UAE North)
Copy data	Succeeded	Copy data	6/14/2026, 6:10:09 AM	17s	AutoResolveIntegrationRuntime (UAE North)
Copy data	Succeeded	Copy data	6/14/2026, 6:09:53 AM	15s	AutoResolveIntegrationRuntime (UAE North)
ForEach File	Succeeded	ForEach	6/14/2026, 6:09:52 AM	2m 52s	
Lookup Parameter Values	Succeeded	Lookup	6/14/2026, 6:09:46 AM	6s	AutoResolveIntegrationRuntime (UAE North)

vi. Verify in ADLS

bronze

Authentication method: Access key (Switch to Microsoft Entra user account)

Search blobs by prefix (case-sensitive)

Showing all 11 items

	Name	Last modified	Access tier
	Calendar	14/06/2026, 06:10:04	
	Customers	14/06/2026, 06:10:22	
	Product_Categ...	14/06/2026, 06:10:40	
	Product_Subcat...	14/06/2026, 06:10:56	
	Products	14/06/2026, 06:11:12	
	Returns	14/06/2026, 06:11:28	
	Sales_2015	14/06/2026, 06:12:02	
	Sales_2016	14/06/2026, 06:12:19	
	Sales_2017	14/06/2026, 06:12:38	
	Territories	14/06/2026, 06:11:44	
	paramvalues.json	11/06/2026, 11:56:32	Cool (Inferred)

Step 5 – Create Microsoft Entra Id

-> This will allow databricks to access the data in the data lake. We will **create a service application** which will access data in the azure data lake.

-> Assign a role (**Storage Blob container**) to this app.

Step 6 – Launch Databricks Workspace (**SILVER LAYER**)

a. Create compute

b. Create a folder in the Databricks workspace

c. Run the below code to access data in the data lake

```
spark.conf.set('fs.azure.account.auth.type.<storage-account>.dfs.core.windows.net',  
'OAuth')  
spark.conf.set('fs.azure.account.oauth.provider.type.<storage-  
account>.dfs.core.windows.net',  
'org.apache.hadoop.fs.azurebfs.oauth2.ClientCredsTokenProvider')  
spark.conf.set('fs.azure.account.oauth2.client.id.<storage-account>.dfs.core.windows.net',  
'<application-id>')  
spark.conf.set('fs.azure.account.oauth2.client.secret.<storage-  
account>.dfs.core.windows.net', 'service credential')  
spark.conf.set('fs.azure.account.oauth2.client.endpoint.<storage-  
account>.dfs.core.windows.net', 'https://login.microsoftonline.com/<directory-  
id>/oauth2/token')
```

→ Replace **<storage-account>** with your storage account name. Similarly, replace **<application-id>** with the App Id generated, **<directory-id>** with the Tenant Id, **service credential** with Value

d. Read the data from all folders/csv file

```
df_cal = spark.read.format('csv')\  
    .option('header', 'true')\  
    .option('inferSchema', 'true')\  
    .load('abfss://bronze@storagesreemala.dfs.core.windows.net/Calendar')  
df_cal.display()
```

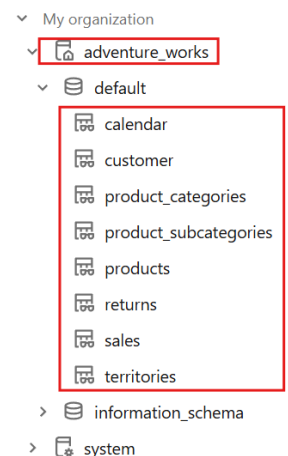
e. Perform transformations as per requirement

Eg – Extract Year and month from Data col in the Calendar table, extract Product SKU and Product name in the Products table.

f. Write the data to MS Azure in Silver container

Step 7 - Create views from the silver data (**GOLD LAYER**)

```
CREATE OR REPLACE VIEW adventure_works.default.Calendar  
AS SELECT * FROM read_files(  
    'abfss://silver@storagesreemala.dfs.core.windows.net/Calendar',  
    format => 'parquet',  
    header => true  
)
```



```
%sql
DESCRIBE EXTENDED adventure_works.default.Sales;
```

_sqldf: pyspark.sql.dataframe.DataFrame = [col_name: string, data_type: string ... 1 more field]

Table ▾ + (

	^B _C col_name	^B _C data_type
)	_rescued_data	string
0		
1	# Detailed Table Information	
2	Catalog	adventure_works
3	Database	default
4	Table	sales
5	Owner	sreemala85@gmail.com
6	Created Time	Sun Jun 14 04:20:49 UTC 2026
7	Last Access	UNKNOWN
8	Created By	Spark
9	Type	VIEW
0	View Text	> SELECT * FROM read_files('abfss://silver@storagesreemala.dfs.core.windows.net/Sales', format => 'parquet', header => true)
1	View Catalog and Namespa...	adventure_works.default
2	View Query Output Columns	[OrderDate, StockDate, OrderNumber, ProductKey, CustomerKey, TerritoryKey, OrderLineItem, OrderQuantity, _rescued_data]